JavaScript Mastery Syllabus

FROM FOUNDATIONS TO MODERN ARCHITECTURE

Phase 1 — Foundations

> **GOAL**
>
> *Get comfortable writing JavaScript. Stop being scared of the syntax.*

Core Topics

- What is JavaScript? History, where it runs (browser, Node.js), why it matters
- Setting up the environment — browser DevTools, VS Code, Node.js installation
- Your first program — `console.log` and other console methods (`warn`, `error`, `table`)
- Comments — single-line (`//`) and multi-line (`/* */`)
- Variables — `var`, `let`, `const` (basic introduction; deeper differences come in Phase 3)
- Naming conventions and rules for identifiers
- Data types — primitive vs non-primitive
  - Primitive: string, number, boolean, null, undefined, symbol, bigint
  - Non-primitive: object (intro only)
- The `typeof` operator
- Type conversion vs type coercion (implicit vs explicit)
- Operators
  - Arithmetic: `+`, `-`, `*`, `/`, `%`, `**`, `++`, `--`
  - Assignment: `=`, `+=`, `-=`, `*=`, `/=`
  - Comparison: `==` vs `===`, `!=` vs `!==`, `>`, `<`, `>=`, `<=`
  - Logical: `&&`, `||`, `!`
  - Ternary operator (`condition ? a : b`)

- String basics — concatenation, template literals, common methods (`length`, `toUpperCase`, `toLowerCase`, `slice`, `substring`, `indexOf`, `includes`, `split`, `trim`, `replace`)
- Number basics — `toFixed`, `parseInt`, `parseFloat`, `Number()`, `isNaN`, the `Math` object
- Conditionals — `if`, `else if`, `else`, `switch`
- Loops — `for`, `while`, `do...while`, `for...of`, `for...in` (brief)
- `break` and `continue`
- Taking user input — `prompt()` in the browser

> **End-of-Phase Mini Projects:**
>
> **Calculator, Temperature Converter, FizzBuzz, Simple Guessing Game.**

Phase 2 — Functions, Arrays & Objects

> **GOAL**
>
> *Learn how to organize logic and data. This is where you start writing "real" programs.*

Functions

- Function declaration vs function expression
- Anonymous functions
- Arrow functions — full syntax, shorthand, implicit return
- Parameters vs arguments
- Default parameters
- Rest parameters (`...args`)
- Return values (single return, early return)
- Functions as first-class citizens — passing functions, returning functions
- IIFE (Immediately Invoked Function Expression) — what and why
- Callback functions — introduction

- Higher-order functions — introduction
- Pure vs impure functions
- Recursion basics

Arrays

- Creating arrays, accessing elements, array length
- Mutating vs non-mutating methods
- Common methods: `push`, `pop`, `shift`, `unshift`, `slice`, `splice`, `concat`, `indexOf`, `includes`, `join`, `reverse`, `sort`
- Iteration methods: `forEach`, `map`, `filter`, `reduce`, `find`, `findIndex`, `some`, `every`
- Array destructuring
- Spread and rest with arrays
- Multi-dimensional arrays

Objects

- Creating objects using object literals
- Accessing properties — dot vs bracket notation
- Adding, updating, deleting properties
- Methods (functions inside objects)
- Nested objects
- Object destructuring (basic and renaming/defaults)
- Spread with objects
- `Object.keys`, `Object.values`, `Object.entries`
- `Object.assign`, `Object.freeze`, `Object.seal`
- Looping through objects

> **End-of-Phase Mini Projects:**
>
> **To-do list (in-memory), Student Grade Tracker, Shopping Cart Logic, Word Frequency Counter.**

Phase 3 — The HOW Begins (Where You Shine)

> **GOAL**
>
> *Understand what is actually happening when JavaScript runs your code. This is the phase that separates good developers from average ones.*

Under the Hood & Execution

- How JavaScript works under the hood — single-threaded, synchronous, interpreted/JIT-compiled
- JavaScript engines — V8 (Chrome, Node), SpiderMonkey (Firefox), JavaScriptCore (Safari)
- Execution Context
  - What is an Execution Context?
  - Global Execution Context (GEC)
  - Function Execution Context (FEC)
  - Two phases: Creation phase (Memory) and Execution phase (Code)
- Call Stack
  - How JavaScript tracks function calls
  - Visualizing the call stack
  - Stack overflow — what causes it
- Hoisting
  - `var` hoisting (initialized as undefined)
  - `let` and `const` hoisting — Temporal Dead Zone (TDZ)
  - Function declarations vs function expressions — hoisting differences
- Scope
  - Global scope
  - Function scope
  - Block scope
  - Lexical scope — what does "lexical" really mean?
  - Scope chain — how JS looks up variables
- Closures
  - What is a closure (the real definition)
  - Step-by-step trace of how closures work
  - Practical use cases — data privacy, function factories, counters, currying

- Common closure interview questions
- Closure + loop classic problem (and why `let` solves it)

> **Recommendation:**
>
> **Pause and revisit Phase 1 and 2 examples with this new lens. The "aha" moments here are what make this phase special.**

Phase 4 — Objects Deeper & `this`

> **GOAL**
>
> *Understand JavaScript's object model — how inheritance really works, and why* `this` *confuses everyone.*

Advanced Object Mechanics

- The `this` keyword
  - `this` in the global scope (browser vs Node)
  - `this` in regular functions
  - `this` in methods
  - `this` in arrow functions (lexical `this`)
  - `this` in event handlers
  - `this` in `setTimeout` / `setInterval`
  - `this` in strict mode
  - `call`, `apply`, `bind` — explicit binding of `this`
- Prototypes
  - What is a prototype?
  - `__proto__` vs `prototype` (the most confusing pair in JS, explained)
  - Prototype chain
  - `Object.create`
  - Inheritance via prototypes

- Constructor functions
  - Using `new` keyword
  - What `new` does behind the scenes (4 steps)
- ES6 Classes
  - `class` syntax
  - `constructor`
  - Instance methods
  - Inheritance with `extends` and `super`
  - Static methods and static properties
  - Getters and setters
  - Private fields (#)

> **Big Reveal:**
>
> **Classes are syntactic sugar over prototypes — proof by example.**

Phase 5 — Asynchronous JavaScript

> **GOAL**
>
> *Master async code. Understand the event loop. Stop being afraid of async/await.*

Concurrency & Async Patterns

- Synchronous vs Asynchronous — what's the difference and why we need async
- Callbacks
  - `setTimeout`, `setInterval`
  - `clearTimeout`, `clearInterval`
  - Callback patterns
  - Callback hell (pyramid of doom) — the problem
- Promises
  - What is a Promise?
  - States: pending, fulfilled, rejected
  - Creating promises with `new Promise`

- Consuming promises — `.then`, `.catch`, `.finally`
- Promise chaining
- Error propagation in chains
- `Promise.all`, `Promise.race`, `Promise.allSettled`, `Promise.any`

- async / await
  - Syntax and rules
  - Error handling with `try/catch`
  - Sequential vs parallel execution
  - Common mistakes (forgetting await, missing try/catch)
- The Event Loop (the second big "wow" moment)
  - Call stack revisited
  - Web APIs / Browser APIs
  - Callback queue (macrotask queue)
  - Microtask queue (Promises, `queueMicrotask`)
  - How the event loop schedules everything
  - Tricky output prediction problems
- Fetch API and HTTP requests — making real network calls
- Introduction to JSON, `JSON.parse`, `JSON.stringify`

Phase 6 — Modern & Practical JavaScript

> **GOAL**
>
> *Round out your toolkit with everything modern JavaScript offers, and prepare for real-world development.*

Modern Tooling & Features

- Modern ES6+ features (deep dive)
  - Destructuring (advanced patterns, nested, with defaults)
  - Spread and rest (advanced use cases)
  - Template literals — tagged templates
  - Optional chaining (`?.`)
  - Nullish coalescing (`??`)

- Short-circuit evaluation patterns (`&&`, `||` as control flow)
- Logical assignment operators (`??=`, `||=`, `&&=`)

- Modules
  - ES Modules — import / export (named exports, default exports, re-exports)
  - Brief mention of CommonJS (`require`, `module.exports`) for Node context
  - Dynamic imports
- Error handling
  - `try`, `catch`, `finally`
  - `throw`
  - Built-in error types (`Error`, `TypeError`, `ReferenceError`, etc.)
  - Creating custom error classes
- Useful built-ins
  - `Map` and `Set` (and `WeakMap`, `WeakSet` briefly)
  - `Date` object
  - Regular expressions (basics — patterns, flags, common methods)
  - Iterators and generators (introduction)
- Storage and persistence
  - `localStorage` and `sessionStorage`
  - Cookies (brief)
- Best practices & Debugging
  - Writing clean, readable code
  - Naming conventions
  - Avoiding common pitfalls
  - Debugging with DevTools — breakpoints, watch, call stack inspection
  - Reading and understanding error messages
- Optional bridge topics
  - DOM manipulation basics — selecting, modifying, events
  - Event delegation and bubbling

> **Capstone Project Ideas:**
>
> **Weather App using Fetch API, Expense Tracker with localStorage, Quiz App, Simple Kanban Board.**

How to Use This Syllabus

💡 **Don't rush.** Each phase has a reason for its position. Skipping ahead means you'll be confused later.

💡 **Code along.** Watching is not learning. Type every example yourself.

💡 **Revisit.** After Phase 3, go back to Phase 1 examples. After Phase 5, revisit Phase 2. You'll see them with new eyes.

💡 **Build the projects.** They tie the concepts together.

💡 **Ask questions.** If something doesn't click, it's almost always because of a missing piece from an earlier phase — find it.

By the end of this course, you won't just know JavaScript. You'll understand it.